



ECOLE POLYTECHNIQUE DE LOUVAIN

Pipeline description

Master's Thesis: Video alignment in a railway infrastructure

Julian GOSSELIN - 20992100 - SINF

Contents

1	Phase 1: Hybrid Alignment (<i>Coarse-to-Fine</i> Approach)	1
1.1	Step 1.1: Macro-synchronization (The DTW Skeleton)	1
1.2	Step 1.2: Micro-alignment	1
1.3	Step 1.3: Global Optimization and Monotonicity Constraint	3
1.4	Step 1.4: Post-processing Smoothing (<i>Anti-Jitter</i>)	3
2	Phase 2: Evaluation Metrics Calculation	3
2.1	Source Distribution (Success Rate)	3
2.2	Temporal Coherence Evaluation	4
2.3	Relative Velocity Analysis	4
2.4	Geometric Quality and Skeleton Correction	4

March 31, 2026

1 Phase 1: Hybrid Alignment (*Coarse-to-Fine* Approach)

The first phase of our pipeline constitutes the core of the alignment algorithm. It relies on a *Coarse-to-Fine* architecture that combines the robustness of global motion analysis with the precision of local feature matching. This phase is divided into four major sub-steps.

1.1 Step 1.1: Macro-synchronization (The DTW Skeleton)

The objective of this step is to create an approximate but globally robust temporal path, acting as a "skeleton" to prevent any long-term drift in the alignment.

- **Feature extraction by optical flow:** Rather than comparing raw pixels, the algorithm extracts a global motion signature for each *frame*. It uses Farneback's dense optical flow algorithm to measure the pixel displacement field. To optimize computation time and filter environmental noise, the image is first resized, the top third (the sky) is ignored, and the *frames* are subsampled (for example, one out of five images). Finally, the image is divided into three vertical regions (left, center, right) to extract motion statistics (mean and standard deviation), thus capturing the parallax effect specific to the train's movement.
- **Alignment by *Dynamic Time Warping* (DTW):** The DTW algorithm is then used to synchronize these two temporal motion sequences. Its internal operation is divided into several key mathematical phases:
 - *Distance matrix:* A Euclidean distance matrix is calculated between the feature vector of each *frame* i of video 1 and that of each *frame* j of video 2.
 - *Dynamic programming and cost matrix:* The algorithm constructs a cumulative cost matrix in order to find the optimal *warping path*. This path connects the indices of the two videos by minimizing the cumulative global distance from the starting point. Movements within this matrix are constrained by temporal monotonicity: time can only move forward, which limits movements to diagonal, rightward, or downward steps.
 - *Step Penalty:* In order to adapt to the fluid and continuous dynamics of a moving train, an adaptive penalty is mathematically added to non-diagonal movements (which correspond to temporal insertions or deletions). This prevents the algorithm from "stuttering" (i.e., associating a single static *frame* of one video with multiple *frames* of the other) and strongly favors a continuous 1:1 synchronization.
 - *Open-End Variant:* Classic DTW forces the path to end exactly in the bottom-right corner of the cost matrix, which would force the ends of the two video sequences to correspond perfectly. Our implementation uses an *Open-End* variant that seeks the optimal exit (the minimum cost) on the last row or the last column of the matrix. This allows the videos to end at different geographic points, respecting the physical reality of our railway dataset where trains can travel different distances for the same recording duration.
- **Interpolation mapping:** The raw alignment path resulting from the DTW is finally transformed into a fluid correspondence dictionary. It provides for each *frame* of video 1, a prediction of the corresponding *frame* index in video 2. The missing intermediate *frames* (due to the initial subsampling of the extraction step) are deduced by linear interpolation between the anchor points validated by the DTW path.

1.2 Step 1.2: Micro-alignment

This step refines the macroscopic prediction of the DTW to obtain an exact, pixel-perfect correspondence. To do this, the pipeline relies on local feature extraction and description algorithms (*Feature Matching*). The system supports three distinct algorithms, offering different trade-offs between speed and precision:

- **ORB (*O*riented *F*AST and *R*otated *B*RIEF):** Proposed as a very fast alternative to SIFT, ORB combines the FAST corner detector (to which it adds an orientation component) and the BRIEF

binary descriptor. Although it is the fastest of the three and invariant to rotation, it is the most sensitive to noise and handles significant scale changes less effectively.

- **BRISK (*Binary Robust Invariant Scalable Keypoints*)**: This algorithm detects points of interest in a continuous scale space using an image pyramid and an AGAST detector. Its binary descriptor uses a circular sampling pattern inspired by the human retina (denser in the center, more spaced out in the periphery). It offers excellent robustness to scale changes (zoom) and blur.
- **AKAZE (*Accelerated-KAZE*)**: Unlike other methods that use Gaussian blur (smoothing the entire image uniformly), AKAZE works in a non-linear scale space (fast explicit diffusion). It allows for noise reduction (such as track ballast) while perfectly preserving structured edges and contours (rails, poles). Its M-LDB binary descriptor captures the local structure with very high precision.

Once the algorithm is chosen, the micro-alignment process revolves around a series of very strict filters and optimizations to avoid false matches:

- **Restricted search window**: For a given *frame* of video 1, the algorithm searches for the best match in video 2 only within a very narrow window centered on the DTW prediction (e.g. ± 10 frames). This drastically reduces computation times and prevents associating distant elements.
- **Binary extraction and matching (Lowe's ratio test)**: When a point of interest is detected on an image, the algorithm assigns it a local "signature" to allow its re-identification in the next image: this is called a *descriptor*. The three algorithms implemented in the code of this pipeline (ORB, BRISK, and AKAZE) have the advantage of natively generating *binary* descriptors, meaning that this signature is a simple sequence of zeros and ones. To compare and match points between video 1 and video 2, the system uses a *Brute Force Matcher* algorithm based on the **Hamming distance**. This distance measures the dissimilarity by counting very rapidly, via an XOR operation, the number of different bits between two signatures. However, the railway environment is full of repetitive patterns (ballast stones, nearly identical railroad ties) that easily deceive the computer. To counter this visual noise, the system extracts the *two* most probable matching candidates in the target image for each point. The **Lowe's ratio test** is then applied: a match is only kept if the distance of the best candidate (D_1) is significantly lower than that of the second (D_2), according to a strict condition (here, $D_1 < 0.75 \times D_2$). This filtering is crucial, as it allows for the massive rejection of ambiguous associations, keeping only the points whose reliability is mathematically indisputable.
- **Spatial coherence filter (Parallax Effect)**: The algorithm geometrically divides each image into two halves (left half and right half). It dictates that a feature detected on the left in video 1 must imperatively correspond to a feature located on the left in video 2 (and vice versa for the right). This filters out physically impossible matches.
- **Geometric Validation (Homography and Scale)**: The remaining matches are submitted to the RANSAC algorithm to calculate a homography matrix and count the number of *inliers* (geometrically valid points). An additional optimization checks the scales extracted from this homography: for railway track videos, the instantaneous time scale should be close to 1.0. If the transformation involves extreme deformation (scale < 0.7 or > 1.3), the *inliers* score is heavily penalized, as this indicates a false positive.
- **Distance Penalty (Scoring)**: To avoid associating aberrant repetitive patterns (such as two distinct but similar-looking railroad ties), the raw *inliers* score is weighted by an exponential decay function based on the temporal distance from the DTW prediction:

$$\text{Weighted_Score} = \text{Inliers} \times e^{-k \cdot |\Delta|} \quad (1)$$

where Δ is the distance between the candidate *frame* and the DTW prediction, and k is a distance penalty constant (e.g., 0.15). The K best candidates (*Top-K*) are kept for each image for the global optimization phase.

1.3 Step 1.3: Global Optimization and Monotonicity Constraint

Micro-alignment produces fairly precise results; however, they can lead to trembling in the final output. (This is the part where the logic can be improved. Currently, the logic is quite simplistic and could work a bit better).

- **Dynamic programming:** The algorithm searches for the optimal path through the matrix of the K best candidates for the entire video.
- **Strict constraint:** The path must be chronologically monotonic, meaning that the index of video 2 can never decrease ($V2_i \geq V2_{i-1}$).
- **Safety net (*Fallback*):** If no local match satisfies the minimum conditions (for example, if the image is black in a tunnel and there are fewer than 4 *inliers*), the algorithm abandons the local visual search and adopts the default DTW prediction as the truth, ensuring no loss of tracking.

1.4 Step 1.4: Post-processing Smoothing (*Anti-Jitter*)

Although global optimization (Step 1.3) mathematically guarantees the absence of temporal regressions, its predictions are inherently discrete (based on integers). This rigidity can generate a visual stuttering effect (*jitter*), where the target image "stagnates" for a few *frames* before abruptly "jumping" to catch up. To ensure a fluid and cinematic progression during the final superposition of the videos, a final smoothing pass is applied. Unlike the previous step which focuses on the absolute **selection** of a *frame*, this step acts as a **cosmetic correction** filter on the already validated results:

- **Weighted moving average filter:** The algorithm deploys a sliding window (e.g., 5 *frames*) along the timeline. For each position, it does not keep the raw value but calculates the average of the surrounding positions.
- **Confidence weighting:** The intelligence of this filter lies in the integration of the quality metric. The surrounding positions do not all carry the same weight: a *frame* validated by a spectacular *RANSAC inliers* score will have a massive gravitational pull on the average, while an uncertain match will be smoothed out in favor of its robust neighbors.
- **Strict rectification validation:** Because applying a smoothing average to a curve can induce slight oscillations, a final algorithmic pass verifies and corrects each final pair to ensure that the smoothing has not mathematically and physically generated any temporal regression (final compliance with the constraint $V2_i \geq V2_{i-1}$).

2 Phase 2: Evaluation Metrics Calculation

Once the process is complete, here are all the tools generated to give me a precise idea of how the algorithm worked, whether it is confident, etc.

2.1 Source Distribution (Success Rate)

This metric evaluates the proportion of *frames* aligned by the local method (AKAZE) compared to those aligned by the safety net (DTW).

- **Percentage *AKAZE Refined*:** Indicates the rate of *frames* where the algorithm was able to find enough points of interest (*inliers* ≥ 4) to ensure a perfect geometric alignment down to the pixel. A high score reflects good visual conditions.
- **Percentage *DTW Fallback*:** Represents areas where the image was too degraded (tunnel, excessive speed) and where the system fell back on the macro-synchronization of the optical flow to avoid losing track.

2.2 Temporal Coherence Evaluation

Aligning railway videos requires perfect temporal fluidity to avoid "jumps" during simultaneous playback.

- **Monotonicity violations:** The system counts the exact number of times the video 2 index moves backward relative to the previous *frame* ($V2_i < V2_{i-1}$).
- **Monotonicity score:** Expressed as a percentage, it quantifies the strict adherence to chronological order. Thanks to the global optimization in step 1.3, this score is designed to approach 100%.

2.3 Relative Velocity Analysis

Given that the trains do not run at exactly the same speed on day D and day D-1, the algorithm calculates the dynamics of relative velocities throughout the journey.

- **Mean velocity and standard deviation:** The instantaneous velocity is calculated as the ratio of progression in video 2 relative to video 1 ($\frac{\Delta V_2}{\Delta V_1}$). The mean indicates which train was overall the fastest, while the standard deviation highlights variations in pace (e.g., different braking).

2.4 Geometric Quality and Skeleton Correction

Finally, the pipeline evaluates the pure precision of *Feature Matching* and its utility compared to simple optical flow.

- **Mean of AKAZE Inliers:** Evaluates the strength of the found matches. A high average number of inliers ensures that the homography matrix calculated by RANSAC is extremely solid.
- **Mean correction (in frames):** Measures the exact difference between the initial prediction provided by the DTW and the final position corrected by AKAZE ($|V2_{final} - V2_{DTW}|$). This makes it possible to precisely quantify the contribution of the method.